

kernel/workqueue.c 横纵分析报告

告

横纵分析法深度研究报告

研究时间：2026-04-30 | 所属领域：Linux 内核异步执行框架 | 研究对象类型：源码文件 / 调度子系统核心实现

作者: GitHub Copilot

研究时间：2026-04-30 | 所属领域：Linux 内核异步执行框架 | 研究对象类型：源码文件 / 调度子系统核心实现

一、一句话定义

`kernel/workqueue.c` 不是一个普通的“队列实现文件”，而是 Linux 内核把 延迟执行、共享线程池、并发控制、同步语义、以及 BH/softirq 兼容层 全部揉在一起后的调度中枢。

如果只看 API，workqueue 很像“把一个函数扔进去，稍后执行”。可一旦往下看这份实现，你会发现它真正解决的问题不是“排队”，而是另一件更难的事：如何在整个内核里，用尽可能少的执行上下文，稳定地跑完尽可能多的异步工作，同时不把 locality、forward progress、flush 语义和历史兼容性搞崩。

这也是为什么 `kernel/workqueue.c` 读起来不像一个容器文件，而更像一个小型调度器。

二、纵向分析：从旧 workqueue 到今天这份 `workqueue.c`

1. 起点不是“队列”，而是内核对异步执行上下文的长期需求

官方文档在开头就把问题说得很直接：内核里有大量场景需要“稍后在另一个执行上下文里做事”，workqueue 是最常见的机制。一个 work item 描述要执行的函数，一个 worker 提供异步执行上下文，队列只是连接两者的组织形式。[来源 1][来源 2]

但早期 workqueue 的矛盾也很早暴露出来了。旧式 MT workqueue 为每个 CPU 维持自己的 worker，ST workqueue 则整个系统只有一个执行上下文。两种设计都不理想：前者浪费线程资源，后者并发度又低得离谱。文档里那句判断很重：资源浪费和并发不足，是同一时期 **workqueue** 的两个硬伤。[来源 1][来源 2]

于是后来才有了 `cmwq`。也就是今天这份 `kernel/workqueue.c` 的精神内核：对外继续保留 workqueue 这个抽象，对内把执行资源统一收束到共享的 worker-pool 上，再让系统按需调节并发。

这一刀的意义非常大。它把 workqueue 从“每个队列带着自己的一组线程”改成了“workqueue 只是工作归属域，线程资源是共享且自动管理的”。这个思路，决定了今天整个文件的骨架。

2. 这份文件里最关键的历史结果：用户可见的 `wq`，和真正执行的 `pool`，被彻底拆开了

如果只看今天的结构体定义，这条历史脉络非常清楚。

`worker_pool` 是底层执行资源池，里面有 `worklist`、`idle_list`、`busy_hash`、`nr_running`、`nr_workers` 等真正与执行相关的状态。它关心的是“谁在跑、谁空闲、队里还有多少活、要不要再拉 worker”。[来源 3]

`workqueue_struct` 则完全是另一种角色。源码注释写得很准确：它是 **externally visible workqueue**，负责通过 `pool_workqueues` 把 work 转发给合适的 `worker_pool`。[来源 3] 也就是说，workqueue 对外是“API 入口”和“语义域”，对内并不是“线程池本体”。

中间那层 `pool_workqueue`，也就是 `pwq`，是整套设计真正聪明的地方。它既知道自己属于哪个 `wq`，也知道自己挂在哪个 `pool` 上，还顺手承接了 `nr_active`、`inactive_works`、flush color、in-flight 统计、mayday 等管理语义。[来源 3]

这三层一拆，整个文件的逻辑就顺了：

1. 外部调用者只和 `workqueue_struct` 打交道。
2. 具体 work 被映射到某个 `pool_workqueue`。
3. 最终真正执行它的是底层 `worker_pool` 里的 worker。

所以今天的 `kernel/workqueue.c`，本质上已经不是“workqueue 的实现”，而是 `wq` / `pwq` / `pool` / `worker` 四层之间的路由与约束系统。

3. 执行路径为什么会长成这样

把这份文件压缩成一句执行链路，其实就是：

`queue_work()` → `__queue_work()` → 选中 `pwq/pool` → `worker_thread()` 取活 → `process_one_work()` 真正执行回调。

真正决定体系气质的是 `__queue_work()`。在这里，系统先根据 CPU、`WQ_UNBOUND`、当前上下文等条件选中目标 `pwq`，再拿到对应 `pool`。[来源 4] 如果这个 work 上一次在另一个 pool 里执行过，而且现在可能还在跑，那么它不会简单地“重新丢到新地方”，而是优先维持 non-reentrancy，尽量继续排到原来的执行环境里。[来源 4]

这一段特别像调度器思维，而不像普通队列思维。普通队列只关心“把元素插进去”，这里则要先处理三个更麻烦的问题：

1. CPU 归属和 locality
2. 是否会和同一个 work 的前一次执行重入冲突
3. `max_active` 限流下，应该直接激活还是先放进 `inactive` 队列

最后一步尤其关键。`__queue_work()` 并不会把所有 work 一股脑都塞进 `pool->worklist`。如果 `nr_active` 已经顶到上限，新 work 会先进入 `pwq->inactive_works`，等后续再被激活。[来源 4] 这意味着 workqueue 的排队并不是单层 FIFO，而是 活跃队列 + 非活跃队列 的双层管理。

等 worker 真把活拿到手，`worker_thread()` 和 `process_one_work()` 又把另一套约束补齐：设置 `current_work/current_func/current_pwq`，清 pending，记账，跑回调，最后再清理状态并减少 in-flight 计数。[来源 5]

这里最能体现 `workqueue.c` 的性格的一点，是它对“执行后遗症”的警惕。`process_one_work()` 在执行前后会检查 lockdep 深度、RCU 深度、atomic 状态，一旦 work 函数泄露了锁、RCU 或原子上下文，直接报 BUG。[来源 5] 这说明 workqueue 不是单纯提供执行机会，它还在替内核当“执行纪律的看门人”。

4. cmwq 的核心不是有 worker，而是会控制 worker

文档里关于 cmwq 的那段表述，我觉得是理解整个文件的钥匙：对于线程池来说，最难的问题不是有没有线程，而是 并发度到底该开多少。cmwq 的目标是维持“最小但足够”的并发度。[来源 1][来源 2]

这句话落到代码里，就是 `worker_pool` 上那套 `nr_running`、`need_more_worker()`、`keep_working()`、`may_start_working()`、`manage_workers()` 的组合拳。

逻辑很朴素，但非常内核：

- 只要 CPU 上已经有 runnable worker，不急着再拉新 worker。
- 一旦最后一个 runnable worker 睡下去，而队列里还有活，就立刻补一个新的执行上下文。
- 空闲 worker 不会立刻被杀，因为保留一些 idle worker 的代价比频繁创建/销毁线程更低。[来源 2] [来源 5]

这就解释了为什么 workqueue 能同时兼顾两个看起来冲突的目标：一边是“不想开太多 kworker”，另一边是“不能让异步工作饿着”。它的解决方案不是静态线程数，而是运行时根据调度行为动态收缩和扩张。

从这个角度看，`kernel/workqueue.c` 真正继承的不是“队列”传统，而是“受约束的执行资源调度”传统。

5. 第二次重要进化：unbound 不再等于“哪里都行”，而是“有弹性的拓扑亲和”

如果说 cmwq 重构解决的是“线程资源怎么共享”，那后面几年 workqueue 的主线改造，解决的就是另一个更细的问题：**unbound workqueue 到底要不要保留 locality。**

文档对 `WQ_UNBOUND` 的表述很直接：它牺牲 CPU locality，换更灵活的执行上下文，尤其适合并发波动大、或者长时间 CPU 密集型工作负载。[来源 2] 但现代内核又不想把 locality 丢得太干净，于是就演化出了 affinity scope。

本仓库的 git 历史能清楚看到这条线：2023 年附近开始出现 `Generalize unbound CPU pods`、`Add multiple affinity scopes and interface to select them`、`Implement non-strict affinity scope for unbound workqueues` 等一串提交，把 unbound workqueue 从“纯粹无绑定”推进成“按 pod / cache / NUMA / system 等不同范围建池”的精细化体系。[来源 6]

这一段演进留下了两个重要结果。

第一，`WQ_UNBOUND` 现在并不意味着彻底放弃局部性。它更像是在说：并发管理交给调度器，但 locality 仍然可以通过 affinity scope 做软约束。

第二，`kernel/workqueue.c` 也因此多了一大块过去没有的拓扑管理逻辑：pod、cpumask、effective cpumask、rescuer affinity、cpuset / housekeeping 联动。这也是为什么今天这份文件会显得“越来越不像一个单纯的执行框架文件”，而更像一个把 CPU 拓扑、热插拔、隔离、回收语义都粘起来的协调层。

6. 第三次重要进化：BH workqueue 要接走 tasklet 的历史包袱

如果说 unbound/pod 是过去两年的“性能与拓扑主线”，那 BH workqueue 则是“历史机制收编主线”。

文档里已经给了非常明确的定义：BH workqueue 仍使用同一套框架，但因为每 CPU 只有一个 BH 执行上下文，所以无需考虑 threaded workqueue 那套并发管理；它本质上是 softirq 的便利接口，并且 BH work item 不能睡眠。[来源 2]

源码在 `line 3701` 的 TODO，把这条路线写得更露骨：

Convert all tasklet users to workqueue and use softirq directly.[来源 7]

这句话的分量其实很大。它不是随手写的 TODO，而是在宣告一条非常明确的技术方向：

1. `tasklet` 还没完全退场，所以当前 `workqueue_softirq_action()` 仍然会被 `tasklet[_hi]action()` 间接调用。[来源 7]
2. 但最终目标不是让 tasklet 永远当入口，而是让 BH workqueue 直接使用 `softirq action`。
3. 一旦所有 tasklet 用户都迁完，这里那句 `need_more_worker()` 的兼容判断也能进一步简化。
[来源 7]

这说明 `kernel/workqueue.c` 的今天，其实处在一个很典型的内核中间态：新框架已经成型，但历史入口还没彻底拔干净。

我觉得这也是理解这份文件最重要的一层语境。很多读者看到这里会以为“为什么 BH workqueue 还和 tasklet 扯在一起，这不是设计不纯吗”。但如果把它放回演进史里看，就会发现这不是不纯，而是在不打碎旧生态的前提下，一点点把旧机制吸进新框架里。这恰恰是内核工程最常见、也最真实的前进方式。

7. flush / drain / cancel：这份文件真正难的，不是执行，而是收尾

很多人第一次读 `workqueue.c`，注意力都放在“怎么排队、怎么唤醒 worker”。但越往后看越会发现，这份文件更难的部分其实是 同步语义。

`__flush_workqueue()` 的注释写得很清楚：它等待的是“调用进入时已经排入的 work 全部完成”，而不是简单地等队列彻底为空。[来源 8] 为了做到这一点，它引入了一整套 color 机制：推进 `work_color` / `flush_color`，让 flush 能区分“这批之前的活”和“后面新来的活”。[来源 8]

`drain_workqueue()` 则比 flush 更强。它会把 workqueue 标记成 `__WQ_DRAINING`，此时只允许链式 queueing，再反复 flush，直到所有 pwq 都空。[来源 4][来源 8] 这是一种很重但很可靠的“最终收束”语义，所以 destroy 前一定会走它。

`cancel_work_sync()` 更有意思。它先尝试把 pending work 从 timer / worklist / pending 状态里抢下来，如果抢不到、说明 work 已经在执行，就转去 `__flush_work(work, true)` 等它跑完。[来源 8] 这一套设计透露出一个很强的工程判断：异步执行不是难在“发出去”，而是难在“我什么时候能确定它真的结束了”。

`kernel/workqueue.c` 对这个问题的答案，不是给你一个模糊的“差不多完了”，而是把 flush、drain、cancel 的边界拆得非常细。

从历史上看，这也是 workqueue 能长期成为内核默认异步机制的重要原因。因为对大多数子系统来说，真正要命的不是“少一个执行器”，而是“收不回来、停不干净、销毁时状态不明确”。

8. 一张压缩时间线

阶段	关键节点	代表意义
旧式 workqueue 时期	每 CPU / 全系统各自持有执行资源	资源浪费与并发不足并存
cmwq 成型	文档明确共享 worker-pool、按需并发	

阶段	关键节点	代表意义
		workqueue 从“带线程的队列”转向“工作归属域”
unbound 精细化	unbound CPU pods 、 affinity scopes 、 non-strict affinity	unbound 从“无 locality”升级为“可调 locality”
BH workqueue 引入	Implement BH workqueues to eventually replace tasklets	用 workqueue 框架收编 tasklet / softirq 半部语义
当前阶段	CACHE_SHARD 默认、rescuer/cpumask/ housekeeping 修补	体系进入性能与边界语义打磨期

这一整条纵线指向同一个事实：kernel/workqueue.c 一直在做“统一”。统一线程资源，统一异步语义，统一 BH 与 threaded 两套执行形式，统一新旧机制之间的迁移路径。

三、横向分析：workqueue.c 在今天的内核里到底站在哪

如果把 kernel/workqueue.c 放到当前 Linux 内核的并发与延迟执行机制里看，它并没有“直接竞品”意义上的对手。更准确地说，它面对的是几个相邻机制：bound/unbound 自己内部的不同路线，threaded 与 BH 的执行上下文差异，以及 tasklet/softirq、kthread_worker 这些邻近方案。

所以横向分析，不是看“谁和它一样”，而是看 workqueue 这个框架到底吸纳了什么，又保留了什么边界。

1. bound vs unbound：一个强调 locality，一个强调弹性

文档对两者的区分非常鲜明。

WQ_PERCPU 适合 CPU locality 重要的场景；work 会绑定到特定 CPU 上。[来源 2]

WQ_UNBOUND 则把 work 放进不绑定特定 CPU 的 worker-pool，让它更像“通用执行上下文提供者”，适合并发需求波动大、或长 CPU 密集型工作负载。[来源 2]

如果只看这层定义，差异好像很简单：一个在本地 CPU 跑，一个全局调度。但真正有意思的是它们在 kernel/workqueue.c 里的实现哲学完全不同。

bound workqueue 的重心是 自动并发管理。worker-pool 通过感知 runnable worker 的数量，决定要不要补 worker。[来源 2][来源 5] 它追求的是“CPU 不空转，但也别平白多开线程”。

unbound workqueue 的重心则不是这套调度钩子，而是 **属性与拓扑选择**。它不再强调 per-CPU 并发控制，而把问题转成“这个 work 应该落到哪个范围的 unbound pool 里”。于是才有后来的 pod、scope、strict / non-strict affinity 这一整条演进线。[来源 6]

说得更直白一点：

- bound 的核心问题是“怎么开得刚刚好”
- unbound 的核心问题是“怎么放得更合理”

2. threaded workqueue vs BH workqueue：同一框架，两种上下文

这是 `kernel/workqueue.c` 特别漂亮的一层统一。

文档一开始就说，work item 可以在 thread context，也可以在 BH (softirq) context 中执行。[来源 1] [来源 2] 也就是说，workqueue 从抽象层面并不执着于“线程”，它更执着的是“把 deferred work 放进一套统一 API 和统一生命周期管理里”。

于是 threaded workqueue 和 BH workqueue 的差别，被压缩成两个维度：

1. 执行上下文能不能睡眠
2. 并发模型要不要复杂管理

threaded workqueue 运行在 kworker 线程里，可以睡眠，因此能承担大量通用异步任务。

BH workqueue 运行在 softirq/BH 语义下，不能睡眠，但依然支持 delayed queueing、flush、cancel 等同类接口。[来源 2]

这非常关键。它意味着 workqueue 不只是“线程池 API”，而是正在努力成为 内核 **deferred execution** 的统一前端。不同上下文只是后端执行器不同，前端接口和大多数管理语义尽量一致。

line 3701 的 TODO 之所以重要，也正是因为说明这种统一还在继续：BH workqueue 还没完全接管 tasklet 遗产，但方向已经定了。[来源 7]

3. workqueue vs tasklet / softirq：新框架不是替代所有历史，而是接管更高层语义

如果把 workqueue 和 tasklet/softirq 放在一起比较，很容易得出一个过于粗糙的结论：workqueue 高级，tasklet/softirq 低级。这个说法不算错，但不够准确。

更准确的说法是：

- softirq 是更底层的执行机制
- tasklet 是建立在历史 softirq 使用方式上的一种封装
- workqueue 则是把“延迟执行”提升到一个更强语义层的统一框架

threaded workqueue 的优势很明显：能睡眠、能共享 worker-pool、能做严格的 flush/drain/cancel、能配合 rescuer 保证 forward progress。[来源 2][来源 8]

BH workqueue 则展示了另一面：它并没有试图“打败 softirq”，而是把 softirq 的执行语义收编进 workqueue 这套更完整的生命周期管理里。[来源 2][来源 7]

这就是为什么文档会把 BH workqueue 定义成“convenience interface to softirq”。它不是否定底层，而是给上层调用者一套更统一、更不容易踩坑的接口。

从技术趋势上看，tasklet 的位置正在变弱，而 BH workqueue 的位置在变强。不是因为 tasklet 完全没法定用，而是因为统一的 **deferred execution** 入口比碎片化的历史接口更值得维护。

4. workqueue vs kthread_worker：共享系统资源 vs 私有执行器

`kthread_worker` 是另一个很值得对照的对象。

如果说 workqueue 是“全内核共享的异步执行基础设施”，那 `kthread_worker` 更像“某个子系统自带的私有执行器”。它通常就是一个专属 kthread 配一个 work_list，顺序地把任务跑完。它的优点是控制感强、语义直观、串行特性清楚；它的代价是共享性差，很多高级语义要自己承担。

而 `kernel/workqueue.c` 的方向正好相反：尽量把公共问题收走，让大部分子系统不要自己维护线程、flush 逻辑、回收语义、rescue 机制、CPU 亲和策略。

有一个很能说明生态位的细节：`pool_workqueue` 的释放本身还会“punt to a kthread_worker”。[来源 3] 这件事很妙，因为它说明 `kthread_worker` 在现代内核里并没有消失，而是退到了辅助角色：当 workqueue 需要一个更私有、更可控的内部执行器时，它会借用 `kthread_worker`，但不会把主舞台让出去。

换句话说，workqueue 并没有消灭其他机制，而是把自己变成了默认层；其他机制则更多在边角场景里提供补充能力。

四、横纵交汇洞察

1. 今天的竞争位置，是被“统一”这条历史路线塑造出来的

如果只横着看，你会觉得 `workqueue.c` 很像一个“四处扩张的框架”：既管 threaded work，又管 BH；既管 bound，又管 unbound；既管 queue，又管 flush/cancel；甚至还开始接 tasklet 的班。

但如果把纵线拉出来，这一切就不是扩张，而是自然结果。

因为 workqueue 从 cmwq 重构那一刻开始，核心命题就不是“做一个更快的队列”，而是“做一个能承接全内核 deferred execution 复杂性的统一层”。

既然命题是统一，那么今天它吸纳 locality、拓扑亲和、rescue、BH、tasklet 迁移，几乎是必然的。

2. 它今天最强的优势，都能追溯到早年的架构拆分

今天 workqueue 最大的优势，我认为有四个：

1. 共享 **worker-pool**，避免每个子系统都自己养线程
2. 自动并发管理，尤其适合短小高频的普通异步任务
3. **flush / drain / cancel** 语义成熟
4. 既能跑 **threaded**，又能逐步把 **BH/softirq** 也统一进来

这四个优势，归根到底都来自同一个历史决定：把 **wq** 和“实际执行资源”拆开。

一旦拆开，workqueue 就不再被单个线程池绑定，而可以演化成“语义容器 + 执行资源路由器”。

3. 它今天最重的包袱，也来自这条历史路线

优势的另一面就是包袱。

因为 workqueue 想统一太多东西，所以 **kernel/workqueue.c** 今天已经是一份高复杂度文件。它同时背着：

- 执行路径
- 同步路径
- rescuer
- hotplug
- unbound affinity
- BH 特判
- 历史兼容

于是它的劣势也很明显：代码面广、语义层次多、读写门槛高。

line 3701 那个 TODO 就是一个很典型的缩影。它说明 BH 路线已经确定，但 tasklet 还没完全退出。这种“中间态”在工程上很正常，但在实现上会把文件继续推向复杂。

说得更狠一点：**kernel/workqueue.c** 的伟大之处和沉重之处，其实是同一件事。它之所以成为默认框架，恰恰是因为它愿意吞下那些别人不想处理的复杂性。

4. 三个未来剧本

最可能的剧本

BH workqueue 继续吃掉更多 tasklet 用户，softirq 入口逐步直接化；unbound affinity 的默认策略继续围绕 cache/cache_shard 做微调；整体方向不是大改框架，而是持续清历史债、修边界、抠性能。

最危险的剧本

统一带来的复杂性继续堆高，导致 `kernel/workqueue.c` 成为越来越难以安全修改的“中心文件”。一旦 BH、hotplug、rescuer、affinity 这几条线在角落里交叉出新 bug，维护成本会上升得很快。

最乐观的剧本

tasklet 彻底退场，BH workqueue 与 softirq 的桥接层明显收敛；unbound 策略进一步稳定，workqueue 成为内核 deferred execution 更彻底的统一入口，调用者只需理解少数明确语义，而不用继续在多套历史接口间摇摆。

五、信息来源

1. 本地源码： `kernel/workqueue.c`
2. `struct worker_pool / pool_workqueue / workqueue_struct`：约 195-397 行
3. `__queue_work()`：约 2275-2391 行
4. `process_one_work()` / `worker_thread()`：约 3200-3479 行
5. BH workqueue TODO 与 softirq 入口：约 3700-3795 行
6. `__flush_workqueue()` / `drain_workqueue()` / `cancel_work_sync()`：约 4040-4550 行
7. 本地文档： `Documentation/core-api/workqueue.rst`
8. threaded/BH 设计说明：约 80-100 行
9. 并发管理与 forward progress：约 117-154 行
10. `WQ_BH` / `WQ_UNBOUND` / `WQ_MEM_RECLAIM` 等 API 语义：约 175-217 行
11. 官方在线文档：<https://docs.kernel.org/core-api/workqueue.html>（访问时间：2026-04-30）
12. 本地 git 历史（`git log --grep='BH workqueues|affinity scope|unbound CPU pods|repatriation|rescuer|tasklet' -- kernel/workqueue.c Documentation/core-api/workqueue.rst`）
13. `4cblef64609f` — `workqueue: Implement BH workqueues to eventually replace tasklets`
14. `84193c07105c` — `workqueue: Generalize unbound CPU pods`

15. 63c5484e7495 — workqueue: Add multiple affinity scopes and interface to select them
16. 8639ecebc9b1 — workqueue: Implement non-strict affinity scope for unbound workqueues
17. 5920d046f7ae — workqueue: add WQ_AFFN_CACHE_SHARD affinity scope
18. 4cdc8a7389d5 — workqueue: set WQ_AFFN_CACHE_SHARD as the default affinity scope
19. 1acd92d95fa2 — workqueue: Drain BH work items on hot-unplugged CPUs
20. 85f0ab43f9de — kernel/workqueue: Bind rescuer to unbound cpumask for WQ_UNBOUND

六、方法论说明

本文采用横纵分析法：纵向上追踪 `kernel/workqueue.c` 从旧式 workqueue、cmwq、unbound 亲和，到 BH workqueue / tasklet 迁移的演进过程；横向上把它放进 Linux 当前 deferred execution 机制中，与 bound/unbound、BH/threaded、tasklet/softirq、kthread_worker 做对照，从而判断它今天真正的生态位。